

Differentiable Graph Neural Hardware/Software Partitioning with Order-Aware Scheduling

Author Name

Affiliation

email@example.com

Draft scaffold, March 2026

Abstract

Hardware/software (HW/SW) partitioning remains difficult because the best partition is rarely the one with the lowest standalone execution cost; it is the one that interacts best with precedence, resource serialization, hardware area limits, and cross-partition communication. Classical exact methods quickly become impractical as graph size grows, while metaheuristics often spend substantial time exploring large binary search spaces. Recent graph-neural approaches improve scalability by turning partitioning into a differentiable graph learning problem, but most formulations still optimize placement more directly than schedule order.

This draft paper studies a repository implementation that extends differentiable graph-based HW/SW partitioning in two stages. The first stage, `diff_gnn`, learns a relaxed hardware-assignment probability for each task node and optimizes a differentiable surrogate of makespan with area and communication penalties. The second stage, `diff_gnn_order`, adds explicit ordering heads, soft permutations produced through Sinkhorn normalization, and an order-aware decode procedure that couples placement and scheduling structure more tightly. The implementation also combines paper-inspired node features, learned edge weighting, differentiable local search, and a queue-based or list-scheduling-style discrete evaluator.

The goal of this manuscript is not to claim a final publishable result, but to provide a realistic six-page research-paper scaffold that mirrors the structure of the 2021 GCN-based HW/SW partitioning paper and the current 2025/2026 internal experiment draft. Using numbers already present in the repository, we show how to present both favorable and mixed evidence: on a fifteen-instance random benchmark summary the ordered model improves mean and median makespan over the baseline, while on a small eleven-node demonstration graph the benefit depends on the exact regularization setting. The resulting draft illustrates one credible paper narrative for later manual rewriting.

1 Introduction

Hardware/software co-design seeks a division of functionality between programmable software and specialized hardware such that the resulting system meets performance, area, and communication constraints. In practice, that division is usually represented as a binary labeling over a task graph: some tasks remain in software for flexibility, while others move to hardware for acceleration. The challenge is that the quality of a partition is determined not only by local node costs but also by how that partition changes the eventual schedule. When a task is moved to hardware, it may shorten its own execution time while simultaneously increasing communication overhead or creating new serialization effects on the software lane. For this reason, partitioning and scheduling are tightly coupled even when they are implemented in separate software modules.

The literature reviewed in the 2021 paper by Zheng *et al.* argues that exact methods such as integer linear programming, dynamic programming, and branch-and-bound are valuable on small instances but become too expensive for large graphs [2]. Metaheuristics such as genetic algorithms, simulated annealing, particle swarm optimization, and related hybrids enlarge the feasible problem size, yet their search still depends on repeated black-box evaluation of discrete candidates. The same paper introduced a graph-convolution-based partitioner, GCPS, to accelerate the search by learning node-level decisions from graph structure and then using schedule feedback to refine them [2]. That idea is compelling because HW/SW partitioning is naturally a node-level decision problem on a directed acyclic graph (DAG).

The current repository continues in that direction but moves from a mostly heuristic GCN pipeline toward a more explicitly differentiable optimization framework. Two implementations are especially relevant:

- `diff_gnn_utils_schedule.py`, which implements a differentiable graph-neural partitioner without explicit ordering heads.
- `diff_gnn_ordering.py`, which augments the model

with ordering logits, soft permutations, and an order-aware makespan surrogate.

The baseline model already goes beyond a naive classifier. It constructs task features from software time, hardware time, area, degree, and paper-inspired gain descriptors; relaxes binary placement through sigmoid or straight-through sampling; optimizes a soft longest-path loss; and then repairs the decoded solution so it respects area constraints. The ordered model adds a second idea: if software tasks are serialized and hardware tasks may also compete for precedence structure, then the model should not learn only *where* tasks go but also *in what relative order* they are likely to execute.

This observation is also consistent with the internal 2025/2026 draft in the repository, which places substantial emphasis on makespan-based objectives such as OPTM, communication-aware scheduling, critical-path list scheduling, event-driven simulation, and shortest-processing-time heuristics. That draft frames the benchmark task graphs as DAGs derived from neural-network models and evaluates candidate partitions through both proxy partition costs and schedule-based objectives. The current repository code mirrors that perspective: discrete solutions can be judged either by a queue-style simulator or by a legacy linear-programming-based makespan evaluator, and additional scheduling heuristics are already implemented in `utils/scheduler_utils.py`.

The purpose of this paper is therefore twofold. First, it documents the current differentiable GNN pipeline in paper form, emphasizing the difference between the baseline partition-only model and the order-aware extension. Second, it provides a concrete formatting scaffold that can later be rewritten manually into a non-AI final manuscript. To support that goal, the paper intentionally uses repository-grounded descriptions and a small set of already available result artifacts rather than inventing a fully new experimental campaign.

The main contributions of this draft are as follows:

- It formalizes the repository’s current HW/SW partitioning problem in a way that is consistent with both the 2021 GCPS paper and the newer internal experiment draft.
- It explains the baseline differentiable placement model and the ordered extension in a unified notation, highlighting exactly what changes when ordering is introduced.
- It demonstrates how to present repository-derived results in a paper narrative, including both positive and negative evidence, rather than forcing a one-sided conclusion.

The rest of the paper is organized as follows. Section II defines the partitioning and scheduling problem. Section

III summarizes related work. Section IV presents the proposed differentiable graph-neural method and the ordered extension. Section V discusses repository-derived experimental results and how they may be presented in a final paper. Section VI concludes, and the appendix maps the paper claims back to implementation artifacts and default settings.

2 Preliminaries

We consider a task graph $G = (V, E)$, where each node $i \in V$ is a task and each directed edge $(i, j) \in E$ denotes a precedence constraint and a potential communication transfer from task i to task j . For every task i , let s_i be the software execution time, h_i the hardware execution time, and a_i the hardware area cost. For every edge (i, j) , let c_{ij} denote the communication cost when the incident tasks are placed on different platforms.

Following both the 2021 paper and the internal draft, the total hardware area is limited. If $x_i \in \{0, 1\}$ denotes whether task i is assigned to hardware ($x_i = 1$) or software ($x_i = 0$), then the feasibility condition is

$$\sum_{i \in V} a_i x_i \leq \alpha \sum_{i \in V} a_i, \quad (1)$$

where $\alpha \in (0, 1]$ is the area-constraint ratio.

2.1 Partition and Schedule Objectives

The internal experiment draft distinguishes between a simple partition cost and multiple schedule-oriented objectives. The simplest partition objective is

$$C_{\text{part}}(x) = \sum_{i \in V} [(1 - x_i)s_i + x_i h_i] + \sum_{(i, j) \in E} |x_i - x_j| c_{ij}, \quad (2)$$

which counts node execution cost and communication across the cut. This form is useful because it is cheap to evaluate, but it does not capture software serialization or more detailed scheduling behavior.

The current repository therefore emphasizes makespan. In discrete evaluation, a partition is passed either to:

- a queue-style simulator implemented in `TaskGraph.evaluate_makespan`, where software tasks execute sequentially and hardware tasks may overlap subject to dependencies, or
- a legacy LP-based evaluator, exposed through `compute_dag_makespan`, for comparison with earlier infrastructure.

Additional scheduling heuristics already exist for event-driven scheduling, communication-aware ordering,

critical-path list scheduling, and shortest-processing-time ordering. This matters because a good paper should distinguish between the *training surrogate* and the *discrete evaluation metric*; the repository code clearly does so.

2.2 Relaxed Assignment

The differentiable models do not optimize binary assignments directly. Instead they produce a relaxed hardware probability $p_i \in [0, 1]$ for each task. The relaxed execution time of task i is

$$\tilde{t}_i(p_i) = p_i h_i + (1 - p_i) s_i, \quad (3)$$

and the relaxed communication cost over an edge (i, j) is approximated by

$$\tilde{c}_{ij}(p) = |p_i - p_j| c_{ij}. \quad (4)$$

This approximation is central to both `diff_gnn` and `diff_gnn_order`, because it makes the cost continuous in the assignment probabilities.

2.3 Graph Features and Edge Information

The repository combines two feature families. The first is a “default” feature profile containing normalized software time, hardware time, area, degree, hardware speedup proxy, area constraint, and hardware scale factor. The second is a “paper” profile inspired by Zheng *et al.*, using hardware gain

$$HG_i = s_i - h_i, \quad (5)$$

and gain per unit area

$$HGP_i = \frac{s_i - h_i}{a_i + \varepsilon}. \quad (6)$$

The implemented hybrid profile, `default_plus_paper`, concatenates both sets of descriptors. Edges can also carry learned or hand-designed weights. In the current configuration, edge attributes include normalized communication cost, $|HG_i - HG_j|$, and $|HGP_i - HGP_j|$, while the effective edge weight can be derived from cosine similarity or learned by a small MLP.

2.4 Problem Statement for This Draft

Given a DAG with node costs, edge communication, and an area budget, we seek a partition and an implied execution order that minimize makespan. The paper focuses on two concrete implementations:

1. **Baseline differentiable partitioning** (`diff_gnn`): learn relaxed placements, decode them into a feasible partition, and evaluate the resulting schedule.

2. **Order-aware differentiable partitioning** (`diff_gnn_order`): learn relaxed placements and additional priority scores that induce soft task orderings during training and decode.

The key technical question is whether learning ordering information jointly with placement improves the quality of the final discrete partition once the schedule is evaluated. The rest of the paper is organized around that question.

3 Related Work

3.1 Exact and Metaheuristic HW/SW Partitioning

Classical HW/SW partitioning has long been formulated as a constrained combinatorial optimization problem. Exact approaches, including mixed-integer linear programming, dynamic programming, and branch-and-bound, provide strong baselines when the graph is small and the constraint model is carefully specified. However, multiple surveys and empirical studies report that these exact methods scale poorly with graph size or with the richness of the scheduling model [1, 3]. Once precedence, communication, and resource constraints are modeled together, exact optimization becomes valuable mainly as a gold standard on smaller instances.

This limitation explains the long-standing use of metaheuristics. Genetic algorithms, simulated annealing, tabu search, particle swarm optimization, differential evolution, and hybrid methods all appear in the HW/SW partitioning literature because they can search larger spaces without full enumeration [6, 7, 8, 9, 10, 11, 12, 13]. The internal draft in this repository follows that tradition by comparing MIP against PSO, DBPSO, CLPSO, CCPSO, SHADE, JADE, GL25, ESA, random assignment, and a greedy heuristic. A useful point from that draft is that the performance of a metaheuristic depends strongly on the evaluation objective: an algorithm that performs well under a coarse proxy cost may not perform equally well when solutions are reevaluated by a scheduling heuristic.

3.2 Graph-Neural Approaches to Partitioning

Graph neural networks (GNNs) are a natural fit for partitioning because the input is already a graph and the output is a node-level decision. The 2021 work of Zheng *et al.* introduced GCPS, a graph-convolution-based partitioner that used node features such as hardware gain and gain per area, cosine-weighted edges, and schedule feedback through list scheduling [2]. Their central claim was that graph convolution can accelerate the search

for large-scale partitions while maintaining or improving quality relative to metaheuristics.

That paper still relied on a training loop whose schedule awareness was partly indirect: the GCN output was converted to labels, a scheduler was run, and the result fed back into training. The current repository goes further by differentiating through surrogate scheduling terms. Instead of treating the partitioner as a heuristic classifier followed by a black-box schedule evaluator, it constructs a continuous objective whose terms are meant to approximate longest-path timing, communication penalties, area usage, and optionally resource ordering.

More broadly, GCN-style models inherit their graph reasoning capability from semi-supervised node classification work such as Kipf and Welling’s graph convolutional network [4]. The use of Sinkhorn normalization and soft permutations in the ordered variant also connects the repository implementation to differentiable ranking and matching methods, especially Gumbel-Sinkhorn relaxations [5]. The novelty in the present context is not the individual components in isolation, but their combination for HW/SW partitioning with schedule-aware decoding.

3.3 Scheduling-Aware Evaluation

The 2025/2026 internal draft is important even though it is not yet a polished paper, because it clarifies the experiment vocabulary used in the repository. It defines a synthetic benchmark model over neural-network-derived DAGs, specifies parameters such as hardware scale factor k , hardware variance l , communication scale μ , and area ratio α , and discusses several scheduling heuristics: OPTM, communication-aware (CA), critical-path list scheduling (CPL), event-driven (ED), and shortest-processing-time (SPT). The repository code contains corresponding scheduling implementations and a queue-style evaluator that is even more tightly integrated with the differentiable methods than the older GCPS pipeline.

This leads to a practical lesson for paper writing. In this problem domain, “better partitioning” is ambiguous unless the evaluation protocol is stated explicitly. A partition may reduce total partition cost while worsening makespan, or match a MIP partition on one metric but differ on another. For that reason, the present draft treats schedule-aware evaluation not as a minor implementation detail, but as part of the method definition.

3.4 Position of This Work

Relative to prior work, the present draft occupies a middle ground:

- it inherits the graph-feature and schedule-aware intuition of GCPS [2],

- it adopts the benchmark and heuristic language of the internal experiment draft,
- and it documents a newer differentiable pipeline in which placement and ordering are optimized jointly.

The most distinctive aspect is the comparison between the repository’s two GNN variants. The baseline `diff_gnn` learns only placement scores, whereas `diff_gnn_order` adds explicit order heads and a resource-aware surrogate. This comparison is narrower than a full benchmark paper, but it is well suited to a method-centric manuscript and to the user’s goal of building a paper template that can later be rewritten by hand.

4 Proposed Method

This section describes the repository implementation as a two-level method family. We first present the baseline differentiable graph partitioner, then the ordered extension, and finally summarize the difference between the two. Throughout, the emphasis is on how the implementation actually behaves rather than on a cleaned-up idealization.

4.1 Baseline: Differentiable Placement Without Explicit Ordering

4.1.1 Graph Encoding

The baseline pipeline begins by converting a `TaskGraph` instance into a `torch_geometric` graph through `_build_torchgeo_data`. The default experimental profile used in the repository is not the original two-feature GCPS encoding. Instead, it often uses `default_plus_paper`, which concatenates:

- normalized software cost, hardware cost, hardware area, graph degree, hardware speedup proxy, area constraint, and hardware scale factor; and
- z-scored paper-inspired descriptors HG and HGP .

This hybrid feature design is important because it preserves the intuition of the 2021 paper while adding implementation-specific context that the older formulation did not encode directly.

Edges are also richer than in a vanilla adjacency matrix. The code can use no edge weights, communication-based weights, cosine similarity, or a learned per-edge transformation. In the configurations most relevant to the repository’s differentiable runs, an edge MLP receives a three-dimensional edge attribute vector composed of normalized communication cost, absolute gain

difference, and absolute gain-per-area difference. The resulting edge scaling modifies message passing and therefore changes which neighbors influence each node’s final embedding.

4.1.2 Placement Network

The baseline model in `diff_gnn_models.py` is a multi-layer GCN ending in a single partition head. Let z_i denote the hardware logit for node i . The model converts these logits into a relaxed hardware probability

$$p_i = \sigma\left(\frac{\gamma z_i}{\tau}\right), \quad (7)$$

where τ is a temperature parameter and γ is an optional logit scaling factor. The code also supports straight-through hard outputs during training, but soft probabilities are the common default.

4.1.3 Differentiable Makespan Surrogate

The baseline loss approximates makespan by a soft longest-path dynamic program over the DAG. Define the relaxed execution and communication terms as in (3) and (4). The baseline surrogate can be summarized as

$$\mathcal{L}_{\text{base}} = \tilde{T}_{\text{path}}(p) + \lambda_a[A(p) - A_{\max}]_+ + \lambda_u(\bar{p} - \rho)^2 + \lambda_e\Phi(p) + \lambda_c\tilde{C}(p), \quad (8)$$

where $A(p) = \sum_i a_i p_i$, $\bar{p}$ is the average hardware usage, ρ is a target hardware fraction, $\Phi(p)$ is an entropy-like binarization term, and $\tilde{C}(p)$ is the expected partition cost. In the implementation, the soft longest-path term is realized by repeated `logsumexp` operations over predecessor states.

Although the baseline objective is differentiable, the final solution must still be discrete. The repository therefore uses a decode-and-repair stage:

1. threshold the relaxed probabilities,
2. repair area violations by greedily removing low-priority hardware nodes,
3. optionally refill unused area with strong hardware candidates,
4. optionally apply differentiable local search (DLS) and a list-scheduling-style postprocess.

This stage is not cosmetic. In practice, it closes much of the gap between a smooth surrogate and the actual discrete schedule metric.

4.2 Ordered Extension: Joint Placement and Scheduling Preference

4.2.1 Additional Heads and Soft Permutations

The ordered model keeps the same graph encoder but replaces the single output head with three heads:

- a partition head that predicts hardware affinity,
- a software-order head producing a priority score for software execution,
- and a hardware-order head producing a priority score for hardware execution.

The software-order head is always active. The hardware-order head is optional and, notably, disabled in the current default configuration through `use_hw_ordering=False`. This is an important implementation detail that a final paper should mention explicitly rather than hiding in an appendix.

Priority vectors are converted into soft permutations through Sinkhorn normalization. For the software lane,

$$P^{\text{sw}} = \text{Sinkhorn}\left(\frac{\phi^{\text{sw}}}{\tau_o}\right), \quad (9)$$

where ϕ^{sw} denotes software-order logits and τ_o is an ordering temperature. Optional Gumbel perturbations can be added, but the fast configurations used in the repository generally disable them.

4.2.2 Pairwise-Order Approximation

The code supports two ways to infer pairwise precedence from the soft permutation: an “exact” matrix expression and a cheaper expected-rank sigmoid approximation. The common setting is `pairwise_mode=rank_sigmoid`, which computes a pairwise “before” probability matrix from expected task ranks. This reduces cost while preserving a notion of relative order. If B_{ji}^{sw} is large, task j is predicted to precede task i on the software resource.

These ordering probabilities are then gated by placement:

$$R^{\text{sw}} = B^{\text{sw}} \odot ((1-p)(1-p)^\top), \quad (10)$$

$$R^{\text{hw}} = B^{\text{hw}} \odot (pp^\top), \quad (11)$$

where $\odot$ denotes elementwise multiplication. In words, a resource-order relation matters only when both tasks actually occupy that resource with high probability.

4.2.3 Order-Aware Makespan Surrogate

The ordered loss extends the DAG-based surrogate with resource-precedence terms. A simplified view is

$$\mathcal{L}_{\text{ord}} = \tilde{T}_{\text{dag+res}}(p, P^{\text{sw}}, P^{\text{hw}}) + \lambda_a[A(p) - A_{\text{max}}]_+ + \lambda_c\tilde{C}(p) + \lambda_{\text{perm}}\Omega(P), \quad (12)$$

where $\Omega(P)$ denotes doubly-stochastic and entropy regularizers over the soft permutation matrices.

Operationally, the code performs iterative refinement. For each node, the start time combines:

- a DAG precedence term from predecessors and cross-partition communication,
- and a resource precedence term derived from other tasks that are likely to execute earlier on the same lane.

This is the main conceptual difference from the baseline. The baseline models dependency timing well, but it does not explicitly learn a schedule order over tasks competing for software or hardware execution slots.

4.2.4 Order-Aware Decode

The ordered model also changes the discrete decode step. Rather than repairing a partition using only placement probabilities, it may blend placement confidence with the difference between hardware and software priority logits. The resulting score preserves nodes that are simultaneously hardware-favored and structurally important to the learned order. However, the code still evaluates multiple decode candidates and keeps the best discrete solution according to the selected metric. This choice is sensible because ordering information can be helpful or harmful depending on the training regime and graph structure.

4.3 Training and Inference Pipeline

The end-to-end pipeline used by both variants is straightforward:

1. build a graph from the task graph instance and selected feature profile;
2. run the GNN to obtain relaxed assignment probabilities, and order logits for the ordered variant;
3. optimize a differentiable surrogate loss with temperature annealing;
4. periodically decode a discrete partition, repair area violations, and evaluate makespan with the chosen metric;

5. keep the best discrete assignment observed during training or final decode.

The practical consequence is worth emphasizing. The final reported partition is not the argmin of the continuous loss. It is the best discrete candidate found after repair and evaluation. This makes the method closer to “differentiable proposal generation plus schedule-aware selection” than to a purely end-to-end differentiable solver. For HW/SW partitioning, that is a reasonable design compromise.

4.4 Why Ordering Can Help

The ordered extension is motivated by a simple failure mode of placement-only learning. Suppose two tasks have similar hardware speedups and similar area demands, but one sits on the critical software chain while the other is off the critical path. A placement-only model may score them similarly, especially when the longest-path surrogate is still a smooth approximation. An order head can inject an additional bias: it can express that one task should appear earlier or later in the software lane relative to its competitors. If that learned order correlates with true schedule criticality, the decode step will preserve more useful hardware assignments.

At the same time, the repository results show that ordering is not automatically beneficial. Soft permutations introduce extra parameters, an extra temperature schedule, and more challenging optimization dynamics. If the order model is under-regularized, weakly supervised, or only partially activated, it can increase runtime without improving the final schedule. A final paper should therefore present ordering as a structured extension with a cost-benefit tradeoff, not as a guaranteed monotonic improvement.

5 Experimental Results

This section is intentionally written as a *draft experimental narrative*. The setup description is taken from the internal HW-SW partitioning draft and the repository code, while the numeric evidence comes from result files already present in the workspace. The section therefore demonstrates how the final paper could be structured, but the numbers should be rerun before any real submission.

5.1 Experimental Setting

The internal draft describes task graphs derived from real neural-network models at tensor-operation granularity. The graph topology is taken from those models, while node and edge costs are assigned synthetically. In particular:

Table 1: Repository-grounded comparison between the baseline and ordered variants.

Component	<code>diff_gnn</code>	<code>diff_gnn_order</code>
Output heads	One partition head producing relaxed hardware logits	One partition head plus separate hardware and software priority heads
Training signal	Soft longest-path surrogate with area, communication, and optional regularizers	DAG-plus-resource surrogate with soft permutations and permutation regularization
Ordering model	Implicit only through final schedule evaluation and local search	Explicit through Sinkhorn-based soft permutations and pairwise-before probabilities
Decode score	Placement probability with optional heuristic speedup blend	Placement probability plus order-aware repair score; evaluates multiple repaired candidates
Postprocess	DLS and optional LSSP-style local search	Same family of postprocessing, but applied after an order-aware decode
Current default limitation	No explicit order modeling	Hardware ordering disabled by default; software ordering remains active

- software costs are sampled from a uniform range;
- hardware costs are sampled around a scaled software cost using a factor k and variance parameter l ;
- hardware area is sampled independently up to a chosen maximum;
- communication costs are sampled relative to the largest software execution time using a scale factor μ .

This setup is also consistent with `TaskGraph.load_graph_from_pydot`, which generates software time, hardware time, area, and communication values from a loaded graph topology.

For the differentiable GNN methods, the most relevant evaluation metrics are:

- **Queue makespan**, produced by the repository’s queue-style simulator;
- **Legacy LP makespan**, produced by the older scheduling backend;
- **Partition cost**, which is useful for diagnosis but not sufficient by itself.

The main comparison in this draft is between `diff_gnn` and `diff_gnn_order`. Classical methods such as GL25, GCPS, and PSO remain valuable reference points, but the purpose here is to understand what the ordering extension contributes beyond the placement-only differentiable model.

5.2 Repository-Derived Example on the Eleven-Node Figure-3 Graph

The repository contains a small but fully analyzable case study in the file `fig3_area18_paper_dls_eval_summary.csv`. The graph has eleven tasks and an area budget of approximately 18.0 units. Table 2 shows the recorded results for

Table 2: Repository-derived results on the eleven-node demonstration graph.

Method	Mksp.	Part.	Area	Cross	Comm.
MIP	34	66	18.0	6	12
Diff-GNN+Order	35	69	15.0	6	14
Diff-GNN	37	66	16.0	5	8

MIP, the baseline differentiable model, and the ordered variant.

This table gives a useful, nontrivial story. The ordered variant improves queue makespan from 37 to 35 relative to the baseline, moving closer to the MIP solution at 34. However, the improvement does not come from a lower partition cost; in fact the recorded partition cost is slightly higher. This is exactly the kind of result one would expect in a schedule-sensitive partitioning problem. A partition can look worse under a static cost proxy yet produce a better actual schedule because it better matches precedence and resource interactions.

Another useful observation is that the ordered model does not simply place more nodes in hardware. In this example it uses *less* hardware area than the baseline while improving makespan. That suggests the gain comes from *where* hardware is used and how the resulting partition interacts with the software schedule, not from saturating the available area budget.

5.3 Mixed Behavior Across Sweep Configurations

A stronger paper should also show cases where ordering does *not* help. The repository already contains such evidence in `fig3_sweep_ranked_2026-02-19.csv`. Two cases are especially informative:

- Under `config_fig3_sweep_v04_minimal`, the baseline records queue makespan 39 while the ordered variant records 35, indicating a clear benefit

Table 3: Aggregate summary over fifteen random instances from repository outputs.

Method	Mean Mksp.	Median Mksp.	Best Cnt.
Diff-GNN+Order	4624.54	5210.00	13
Diff-GNN	4921.82	5423.63	4
GL25	6653.44	5978.47	0
GCPS	6908.76	8369.32	0

from the extra order model.

- Under `config_fig3_sweep_v02_noreg`, the baseline records queue makespan 35 while the ordered variant records 37, indicating a regression.

The same sweep also shows the runtime cost of the ordered model. For example, under `v04_minimal`, the baseline requires about 13.8s whereas the ordered model requires about 26.8s. Under `v08_order_strong`, the ordered model takes more than 64s and still does not beat the best baseline setting. The lesson is that ordering changes both the optimization landscape and the wall-clock budget. A careful final study should therefore report quality and runtime together.

These mixed results are not surprising after inspecting the code. The ordered model introduces extra temperatures, soft permutations, and pairwise-order computations. At the same time, the current default configuration keeps `use_hw_ordering=False`, so only the software-order head is fully active in the loss. The method is therefore best viewed as a partially activated order-aware extension whose benefit depends on graph structure and regularization.

5.4 Aggregate Comparison on a Fifteen-Instance Random Benchmark

The repository also contains an aggregate summary in `shared_random_15_20260226_070718_summary.csv`. Table 3 reports the values for the two GNN variants and two older baselines.

Here the ordered model is more clearly favorable. It improves mean makespan by roughly 6% and median makespan by about 4% relative to the placement-only differentiable model, and it wins thirteen out of fifteen recorded cases. These numbers are not enough for a polished claim because the exact benchmark generation and repetition protocol should still be rerun from scratch. Still, they illustrate how a final paper can argue that the ordering extension is valuable on average even if it is not uniformly dominant on every single small instance.

This aggregate table also suggests a broader narrative. Both differentiable GNN variants outperform older heuristic baselines in the summary, which supports the general premise that graph-based learned surrogates are promising for large combinatorial partitioning spaces.

Within that family, the order-aware variant appears to offer a second layer of improvement by modeling schedule structure rather than only placement.

5.5 Interpretation

Taken together, the repository results suggest three defensible conclusions for a final manuscript.

First, schedule-aware partitioning should be evaluated by actual schedule metrics, not by partition cost alone. The eleven-node example already shows a partition-cost/makespan mismatch.

Second, the ordered extension has a meaningful upside. On the aggregate fifteen-instance summary it improves both mean and median makespan, and on at least some small controlled settings it closes part of the gap between the baseline differentiable model and MIP.

Third, ordering is not free. It increases runtime, enlarges the hyperparameter space, and can regress under weak regularization or mismatched settings. This makes an ablation study essential. In a final experimental section, the following tables would be particularly useful:

- with versus without ordering under identical feature and postprocess settings;
- with software ordering only versus software+hardware ordering;
- queue metric versus legacy LP metric;
- no postprocess versus DLS versus DLS+LSSP.

5.6 Recommended Final Experimental Structure

For later manual rewriting, a full version of this section could be organized as follows:

1. **Setup.** Describe graph sources, the (k, l, μ, α) cost model, seeds, and train/eval metrics.
2. **Method ablations.** Compare `diff_gnn` against `diff_gnn_order`, then switch on or off paper features, learned edge weights, DLS, and hardware ordering.
3. **Comparisons to baselines.** Add GCPS, GL25, PSO-family methods, and MIP where tractable.
4. **Runtime analysis.** Report wall-clock time and memory use.
5. **Schedule diagnostics.** Include cross-edge counts, communication delay, area usage, and critical-path behavior for representative graphs.

That structure is consistent with the repository and with the two reference PDFs. It also gives the user a practical template for rewriting the experiments later in their own wording and with their own rerun data.

6 Conclusion

This draft paper presented a structured description of a differentiable graph-neural HW/SW partitioning pipeline grounded in the current repository. The baseline model, `diff_gnn`, learns relaxed hardware assignments and optimizes a differentiable makespan surrogate. The ordered extension, `diff_gnn_order`, augments that formulation with explicit priority heads, soft permutations, and order-aware decode logic so that placement and scheduling preference are modeled jointly.

The repository artifacts already support a credible draft conclusion: ordering is promising but not automatically dominant. On a fifteen-instance random summary it improves mean and median makespan over the baseline, yet on small sweep cases its benefit depends on configuration and comes with additional runtime. That is a useful and publishable type of result if documented carefully.

As a scaffold, this manuscript shows one reasonable way to turn the code, the 2021 GCN paper, and the newer internal partitioning draft into a six-page research narrative. The next step for a final human-written paper is to rerun the experiments cleanly, tighten the claims, and rewrite the prose in the author’s own voice while preserving the overall structure.

A Implementation and Reproducibility Notes

A.1 Mapping Paper Concepts to Repository Files

Table 4 maps the main paper claims to the concrete implementation files inspected for this draft. Including such a table in an appendix is often useful during paper writing because it prevents the method section from drifting away from the actual code.

A.2 Representative Default Hyperparameters

The repository contains many configuration options, but a reader only needs a small subset to understand the main method. Table 5 lists the defaults that are most visible in the current differentiable path.

Table 4: File-to-method mapping used to build this draft.

File	Main role in the draft
<code>diff_gnn_utils_scheduler.py</code>	Baseline differentiable placement model, feature construction, relaxed assignment, decode-and-repair, differentiable local search, and discrete evaluation hooks
<code>diff_gnn_ordering.py</code>	Ordered extension with additional priority heads, Sinkhorn permutations, resource-aware loss, and order-aware decode
<code>diff_gnn_models.py</code>	Neural architectures for the placement-only and ordered models
<code>task_graph.py</code>	Task-graph representation, random cost generation, area violation checks, and queue-style makespan evaluation
<code>scheduler_utils.py</code>	Event-driven, critical-path, shortest-processing-time, and communication-aware scheduling heuristics
<code>HW:SW2021.pdf</code>	Prior GCN-based HW/SW partitioning paper and terminology source
<code>2026 HW_SW draft PDF</code>	Internal benchmark framing, objective vocabulary, and baseline-method context

A.3 Practical Notes for Rewriting

When this draft is rewritten manually, the following edits would make the final paper stronger:

- replace the placeholder author block and title with the actual project naming convention;
- rerun all experiments with a controlled seed protocol and identical train/eval metrics across methods;
- convert the draft’s repository-derived tables into clean tables generated directly from the rerun outputs;
- explicitly state whether the final version enables hardware ordering or keeps software-only ordering;
- add figures showing example schedules, critical paths, or partition visualizations if needed.

One subtle but important point deserves repetition. The current ordered implementation can outperform the baseline even when `use_hw_ordering=False`. In that regime, the benefit comes from the software-order head, the changed loss, and the order-aware decode machinery, not from a fully symmetric ordering model over both lanes. A final paper should make this distinction clear to avoid overstating what was actually evaluated.

Table 5: Representative default settings exposed in the current code.

Setting	Baseline	Ordered
Hidden dimension	256	256
GNN layers	3	3
Dropout	0.2	0.2
Feature profile	configurable, often hybrid	default_plus_paper
Edge weighting	optional, often learned MLP	often learned MLP
Selection metric	queue by default	queue by default
Postprocess mode	hybrid defaults available	hybrid defaults available
Ordering temperature	not used	1.0 $\rightarrow$ 0.2
Sinkhorn iterations	not used	12 in fast settings
Hardware ordering	not applicable	disabled by default

A.4 Suggested Appendix Material for a Final Version

If the final manuscript grows into a full paper, the appendix could also contain:

- the exact YAML configuration used for each ablation;
- additional per-instance results for the random benchmark set;
- sensitivity plots over k , α , and communication scale;
- a complexity discussion comparing one forward-backward pass of `diff_gnn` and `diff_gnn_order`.

Those additions are unnecessary for the present scaffold, but this appendix section shows where they would fit naturally.

References

- [1] P. Arató, Z. Á. Mann, and A. Orbán, “Algorithmic aspects of hardware/software partitioning,” *ACM Transactions on Design Automation of Electronic Systems*, vol. 10, no. 1, pp. 136–156, 2005.
- [2] X. Zheng, S. Liang, and X. Xiong, “A hardware/software partitioning method based on graph convolution network,” *Design Automation for Embedded Systems*, vol. 25, pp. 325–351, 2021.
- [3] N. Hou, X. Yan, and F. He, “A survey on partitioning models, solution algorithms and algorithm parallelization for hardware/software co-design,” *Design Automation for Embedded Systems*, vol. 23, no. 1–2, pp. 57–77, 2019.
- [4] T. N. Kipf and M. Welling, “Semi-supervised classification with graph convolutional networks,” in *Proc. ICLR*, 2017.
- [5] G. Mena, D. Belanger, S. Linderman, and J. Snoek, “Learning latent permutations with Gumbel-Sinkhorn networks,” in *Proc. ICLR*, 2018.
- [6] J. Kennedy and R. Eberhart, “Particle swarm optimization,” in *Proc. IEEE International Conference on Neural Networks*, 1995, pp. 1942–1948.
- [7] J. Kennedy and R. C. Eberhart, “A discrete binary version of the particle swarm algorithm,” in *Proc. IEEE International Conference on Systems, Man, and Cybernetics*, 1997, pp. 4104–4108.
- [8] J. J. Liang, A. K. Qin, P. N. Suganthan, and S. Baskar, “Comprehensive learning particle swarm optimizer for global optimization of multimodal functions,” *IEEE Transactions on Evolutionary Computation*, vol. 10, no. 3, pp. 281–295, 2006.
- [9] X. Li and X. Yao, “Cooperatively coevolving particle swarms for large scale optimization,” *IEEE Transactions on Evolutionary Computation*, vol. 16, no. 2, pp. 210–224, 2012.
- [10] R. Tanabe and A. Fukunaga, “Success-history based parameter adaptation for differential evolution,” in *Proc. IEEE Congress on Evolutionary Computation*, 2013, pp. 71–78.
- [11] J. Zhang and A. C. Sanderson, “JADE: adaptive differential evolution with optional external archive,” *IEEE Transactions on Evolutionary Computation*, vol. 13, no. 5, pp. 945–958, 2009.
- [12] C. García-Martínez, M. Lozano, F. Herrera, D. Molina, and A. M. Sánchez, “Global and local real-coded genetic algorithms based on parent-centric crossover operators,” *European Journal of Operational Research*, vol. 185, no. 3, pp. 1088–1113, 2008.
- [13] P. Siarry, G. Berthiau, F. Durdin, and J. Haussy, “Enhanced simulated annealing for globally minimizing functions of many-continuous variables,” *ACM Transactions on Mathematical Software*, vol. 23, no. 2, pp. 209–228, 1997.
- [14] J. Kotary, Z. Mehrab, R. Meyur, and S. S. Das, “HW-SW Partitioning,” internal draft report, May 2025.